

Competitive Programming

Section 69: Master+ Structures And Trees Mixed Contest

Competitive Programming Course

Table of contents

1	Contest contract	3
2	Problem set	4
3	Proof checkpoints	5
4	How to identify the boundary before choosing a template	6
5	A discovery walkthrough: Beacon Distance Sum	7
6	A discovery walkthrough: Weighted Marked Pairs	8
7	Contest ordering and validation	9
8	Implementation checklist	10
9	Exercises	11

1 Contest contract

Six original problems cover persistence, rollback, ordered HLD, centroid decomposition, virtual trees, and small-to-large merging. Suggested time is five hours.

Before coding, write:

What past or tree boundary must remain observable?

Which information is immutable, undoable, or query-local?

Does order matter when chunks are combined?

What is the independent tiny oracle?

2 Problem set

Problem	Title
A	Snapshot Range Rank
B	Seasonal Component Size
C	Route Matrices
D	Beacon Distance Sum
E	Weighted Marked Pairs
F	Subtree Mode Profile

3 Proof checkpoints

- A: why do two ancestor-version roots represent exactly $[l, r]$?
- B: why is the rollback DSU at a time leaf the complete current graph?
- C: which chunks must be reversed, and in which order are right chunks joined?
- D: which centroid branch causes double counting, and why does subtracting it leave each active vertex once?
- E: how many weighted pairs cross one virtual edge?
- F: why can a color record move only logarithmically many times?

4 How to identify the boundary before choosing a template

The six problems are deliberately written so that several tree techniques look plausible. Use the operations to eliminate the wrong boundaries.

Evidence in the statement	Boundary that must survive
a query names an old branch and a subrange of its history	two persistent prefix roots
graph edges appear and disappear over time vertex operators are multiplied along an oriented path	rollback state along a time-tree path ordered HLD chunks
active vertices affect distance sums from arbitrary queries	centroid totals minus one branch
a query contains a small temporary marked set	a virtual tree for that query only
every rooted subtree asks for a frequency profile	one mergeable map per unfinished subtree

Do not begin with “this is a centroid problem.” Begin with “what information can still interact with future input?” The technique is the implementation of that boundary.

5 A discovery walkthrough: Beacon Distance Sum

A first idea is to keep all active beacons and compute every distance on a query. This is linear per query. Keeping only the nearest beacon is insufficient because the requested operation is a sum.

At a centroid c , the contribution of a set of beacons whose paths from u pass through c separates into two aggregates:

```
sum(dist(u,c) + dist(c,x))  
= count * dist(u,c) + sum_dist_to_c.
```

This is the bottleneck: those two numbers are sufficient, but the total at c also contains beacons in the same centroid branch as u , whose paths may not pass through c . The smallest repair is not a new decomposition—it is one subtotal per branch. Add the centroid total, subtract u 's branch, and repeat over centroid ancestors.

The proof then charges each active beacon to the first centroid that separates it from u . This “find the first boundary where two objects separate” argument is reusable far beyond this particular sum.

6 A discovery walkthrough: Weighted Marked Pairs

Enumerating all marked pairs is quadratic in k . Reverse the summation: instead of asking which edges each pair uses, ask how many pairs use one edge. If a virtual edge separates coefficient sum s from total coefficient sum T , expansion gives

$$s * (T-s) = \text{sum of } c_i * c_j \text{ over all marked pairs separated by the edge.}$$

Multiplying by the edge length charges every compressed original edge once to every path using it. A virtual tree is then forced by the query shape: the input names only k vertices, so processing all n vertices would ignore the most informative constraint.

7 Contest ordering and validation

A suggested five-hour order is A, B, C, F, E, D. The first three expose whether your persistent, rollback, and directional-HLD templates are reliable. F is mostly an amortized merge. E and D require the most careful counting proofs.

For every problem, prepare a conceptually different tiny oracle:

- A copies the complete version sequence and sorts the requested slice.
- B rebuilds the active graph and runs BFS.
- C explicitly lists the path and multiplies matrices one vertex at a time.
- D and E precompute all-pairs distances on a tiny tree.
- F counts every subtree color from scratch.

Random testing is especially valuable here because ordering and subtraction bugs often satisfy samples. Generate chains, stars, balanced trees, duplicate values, repeated toggles, ancestor-related marked sets, and noncommuting matrices.

8 Implementation checklist

- Are all version and vertex indices converted exactly once?
- Does every rollback DFS exit restore the snapshot it took on entry?
- Are upward and downward HLD chunks accumulated on opposite sides?
- Does a centroid sum query subtract precisely the branch containing u ?
- Are LCAs added before constructing the virtual-tree stack?
- Does smaller-to-larger merge preserve the larger container by reference?
- Can a product, distance sum, or coefficient product overflow 32 bits?

9 Exercises

Solve all six blind. During the upsolve, reproduce the proof checkpoint before reading code. A correct template attached to the wrong invariant is still a wrong solution.